Algoritmo de Cifrado de Una Vía

Informe

Juan David Romero Morales – 20222020102

Jaider Alejandro González Rojas - 20242020075

Jimmy Alejandro Granados Becerra - 20251020034

Juan Sebastian Gutiérrez Poveda - 20251020199

Universidad Distrital Francisco Jose de Caldas

Ciencias de la Computación II – Grupo 020-84

Docente: Simar Enrique Herrera

Bogotá D.C.

19 De Septiembre De 2026

Tabla de Contenido

Pág.

1. Introducción 3

2. Objetivos 4

2.1. Objetivo General 4

2.2. Objetivos Específicos 4

3. Marco Teórico y Conceptos Clave 5

3.1. Explicación Matemática por Fases 5

3.2. Glosario 6

4. Explicación del Código Desarrollado 9

4.1. Funcionamiento General 9

4.2. Detalle Profundo de las Funciones 9

4.3. Flujo de Ejecución del Sistema 12

5. Pruebas 13

6. Conclusión 15

1. Introducción

LINK DEL REPOSITORIO: https://github.com/Jimmy86gb/CC2_GR5_020_84.git

Ruta de Entrega: 1ER_CORTE/4_Cifrado_De_Doble_Via/Entrega

Este proyecto busca desarrollar una herramienta criptográfica sencilla que permita entender de forma práctica cómo funciona el cifrado asimétrico y algunos mecanismos relacionados con la protección de la información. El programa fue desarrollado en C++ y utiliza conceptos del algoritmo RSA para generar claves públicas y privadas, cifrar mensajes y posteriormente recuperarlos mediante la clave correspondiente.

Además del cifrado, el programa incluye una firma digital basada en una función hash propia inspirada en la serie BBP, con el fin de comprobar si el mensaje recibido conserva su contenido original. También se implementa una técnica de inyección de ruido, en la que se agregan caracteres adicionales entre los datos reales del mensaje. De esta manera, el receptor debe identificar correctamente la forma en que fue construido el paquete antes de recuperar el texto original.

Con este desarrollo no se busca reemplazar los algoritmos criptográficos utilizados actualmente, sino aplicar de manera práctica conceptos como aritmética modular, números primos, máximo común divisor, exponenciación modular, funciones hash y generación de claves, integrándolos dentro de un mismo programa.

2. Objetivos

2.1. Objetivo General

Desarrollar una herramienta criptográfica en C++ que permita cifrar, descifrar y verificar mensajes mediante el uso de claves asimétricas, firma digital, una función hash personalizada e inyección de ruido.

2.2. Objetivos Específicos

- Implementar la generación de claves públicas y privadas utilizando conceptos matemáticos propios del algoritmo RSA.
- Aplicar exponenciación modular para realizar los procesos de cifrado y descifrado de los caracteres del mensaje.
- Diseñar una función hash inspirada en la serie BBP que permita obtener un valor representativo del contenido del mensaje.
- Implementar un sistema de firma digital que permita verificar si el mensaje recibido coincide con el mensaje enviado.
- Incorporar caracteres de ruido dentro del mensaje cifrado para agregar una capa adicional de ofuscación durante la transmisión.
- Integrar todos los procesos dentro de una herramienta de consola que permita generar claves, enviar mensajes cifrados y recuperar mensajes recibidos.

3. Marco Teórico y Conceptos Clave

3.1. Explicación Matemática por Fases

- **Fase 1: Creador de Claves (Generación RSA)**
    - **Conceptos matemáticos aplicados:** Números Primos, Aritmética Modular, Función Totiente de Euler (phi) y Algoritmo de Euclides Extendido.
    - **¿Por qué funciona?:** La seguridad se basa en una "función trampa" (un problema matemático fácil de hacer, pero casi imposible de deshacer). Es muy fácil multiplicar dos números primos gigantes para obtener un resultado, pero si a una computadora solo le das el resultado, le tomaría años adivinar cuáles fueron esos dos primos originales. Este concepto permite calcular el "Módulo" (n). Luego, mediante el Algoritmo de Euclides Extendido, calculamos una "Clave Privada" (d) que actúa como el inverso matemático perfecto de la "Clave Pública" (e).
    - **¿Por qué es distinto a alternativas comunes?:** A diferencia de la criptografía simétrica (como AES, donde usas la misma contraseña para bloquear y desbloquear un archivo), este modelo matemático permite revelar tu Clave Pública al mundo entero para que te envíen mensajes, pero solo tú (con la Clave Privada, que es la solución inversa) puedes leerlos.

- **Fase 2: Cifrador y Generación de Firma**
    - **Conceptos matemáticos aplicados:** Exponenciación Modular (C = M^e mod n) y Series Matemáticas (Fórmula BBP para el número Pi).
    - **¿Por qué funciona?:** Para cifrar, el programa toma cada letra y la eleva a la potencia de la Clave Pública, extrayendo el residuo de la división (módulo). Esta operación matemática revuelve el número original. Para la firma digital, el programa calcula un "Hash" (huella dactilar) aplicando sumatorias de la fórmula BBP (Bailey–Borwein–Plouffe, usada en matemáticas para calcular el enésimo dígito hexadecimal de Pi). Esta fórmula genera un caos numérico pseudo-aleatorio basado en cada letra del mensaje. Luego, ese número gigante se firma con la Clave Privada del emisor.
    - **¿Por qué es distinto a alternativas comunes?:** Normalmente, los sistemas

modernos usan algoritmos estándar como SHA-256 para firmas y técnicas de relleno estandarizadas (como PKCS1) para ocultar patrones. Este programa utiliza una inyección de "ruido activo" generada por números aleatorios (rand) creando letras falsas, y usa una función Hash personalizada basada en el cálculo del número Pi, lo cual es altamente inusual y le da una capa de ofuscación artesanal y caótica.

- **Fase 3: Descifrador y Verificación de Firma**
    - **Conceptos matemáticos aplicados:** Teorema de Euler / Pequeño Teorema de Fermat (M = C^d mod n) y Esteganografía básica.
    - **¿Por qué funciona?:** Gracias al teorema matemático subyacente del sistema RSA, si tomas un número cifrado (C) y lo elevas a tu Clave Privada (d) usando aritmética modular, las matemáticas "cancelan" mágicamente el efecto de la Clave Pública y devuelven el número original (M). En la firma, el destinatario toma la huella cifrada recibida y la "desbloquea" usando la Clave Pública del emisor. Si la huella desbloqueada coincide con el contenido del mensaje, significa que matemáticamente fue firmado por la persona correcta y nadie alteró el texto.
    - **¿Por qué es distinto a alternativas comunes?:** Lo diferente aquí es cómo se reconstruye el mensaje antes de verificar la firma. El receptor tiene que descifrar un primer "bloque oculto" que le indica cuántas letras basura fueron inyectadas por cada letra real. Esto mezcla criptografía fuerte (cifrado matemático) con esteganografía (ocultar un mensaje entre la basura). Si el bloque inicial se decodifica en un número sin sentido, el programa sabe que está ante un lector equivocado y no procesa el salto de ruido.

3.2. Glosario

- **Tipos de Variables Utilizadas**
    - **long long:** Enteros con precisión extendida. Son críticos en criptografía para manejar números gigantes y evitar el desbordamiento de enteros (un error informático que ocurre cuando un número es demasiado grande para el espacio de memoria asignado en el sistema).

- **int:** Enteros de tamaño estándar. Usados en opciones sencillas del menú y conteos pequeños.
- **string:** Clase para el manejo de cadenas de texto (oraciones o palabras completas).
- **double:** Números fraccionarios o con decimales. Usados en los cálculos matemáticos finos de la fórmula BBP.
- **size_t:** Entero sin signo (que nunca puede ser negativo). Es la medida de longitud que devuelve el sistema al medir cadenas y se usa para recorrer el texto sin errores.
- **char:** Carácter simple (una sola letra o símbolo). Permite manipular, inyectar y descifrar letras una por una.
- **bool:** Operador lógico de verdadero/falso. Actúa como un interruptor para validar si se superó la prueba de descifrado.

- **Librerías Importadas (Includes)**
  - **<iostream>:** Maneja la entrada (leer lo que escribes) y salida (mostrar texto en pantalla).
  - **<string>:** Habilita las herramientas avanzadas para manipular texto en C++.
  - **<cmath>:** Agrega funciones matemáticas complejas (como potencias o valores absolutos).
  - **<sstream>:** Flujos de texto que permiten traducir o transformar números de formato normal a formato hexadecimal (sistema de numeración base 16).
  - **<iomanip>:** Herramientas estéticas para rellenar con ceros o dar formato ordenado al texto cifrado que aparece en pantalla.
  - **<limits>:** Conoce las capacidades máximas de la memoria para ayudar a vaciar la basura retenida al escribir.
  - **<cstdlib> y <ctime>:** Generan números aleatorios impredecibles basados en el reloj (la hora actual) de tu computadora, fundamentales para crear el "ruido".

- **Palabras Reservadas Clave**
  - **const:** Una promesa de seguridad que asegura que una variable de entrada no será

alterada ni corrompida accidentalmente por la función.

- **void:** Indica que una función realiza una tarea pero no devuelve ningún cálculo al sistema al finalizar.
- **while / for:** Bucles (ciclos repetitivos) para hacer tareas constantes.
  return: El resultado final que una función le entrega a la parte del programa que la llamó.

4. Explicación del Código Desarrollado

4.1. Funcionamiento General

El código implementa una herramienta criptográfica en consola escrita en C++ llamada "brap". Permite la generación de claves asimétricas (un sistema de seguridad con una clave pública para encriptar y otra privada para desencriptar) basadas en el algoritmo RSA (el método matemático estándar en la industria para proteger datos). Su principal novedad es que, durante el cifrado, inyecta "ruido activo" (datos o caracteres basura insertados a propósito para confundir a posibles intrusos) intercalando el texto real con estos caracteres generados al azar. Además, incluye un sistema de firma digital y validación utilizando una función hash (una huella digital matemática y única de un archivo o texto) personalizada e inspirada en la serie BBP para Pi (una fórmula matemática usada originalmente para calcular decimales del número Pi).

4.2. Detalle Profundo de las Funciones

- **aHexadecimal()**
    - **Función:** Convierte un número entero puro a una cadena de texto en formato hexadecimal asegurando 8 caracteres de largo.
    - **Recibe:** long long valor
    - **Devuelve:** string

- **desdeHexadecimal()**
    - **Función:** Realiza la operación inversa a la función anterior, traduciendo texto hexadecimal al valor numérico real.
    - **Recibe:** const string& hexStr
    - **Devuelve:** long long

- **calcularMCD()**
    - **Función:** Calcula el Máximo Común Divisor entre dos números utilizando el algoritmo clásico de Euclides.
    - **Recibe:** long long a, b

    - **Devuelve:** long long

- **euclidesExtendido()**
    - **Función:** Ejecuta una operación matemática para hallar el inverso multiplicativo modular (tu clave privada).
    - **Recibe:** long long e, phi
    - **Devuelve:** long long

- **generarClavesAsimetricas()**
    - **Función:** Director de orquesta para crear un nuevo usuario usando números primos grandes.
    - **Recibe:** long long p, q; &n, &e, &d
    - **Devuelve:** void

- **expModular()**
    - **Función:** Aplica la exponenciación modular usando atajos de bits para no sobrecargar el procesador.
    - **Recibe:** long long base, exp, mod
    - **Devuelve:** long long

- **calcularHashBBP()**
    - **Función:** Crea una firma digital usando fracciones inspiradas en la fórmula BBP de Pi.
    - **Recibe:** const string& texto, long long modulo
    - **Devuelve:** long long

- **cifrarMensajeHex()**
    - **Función:** Motor de encriptación y ofuscación inyectando ruido aleatorio.
    - **Recibe:** const string& mensaje, clavePublica, modulo
    - **Devuelve:** string

- **descifrarMensajeHex()**
    - **Función:** Motor de recepción que lee el salto secreto y evade el ruido para reconstruir el mensaje.
    - **Recibe:** const string& cifradoHex, clavePrivada, modulo
    - **Devuelve:** string

- **limpiarBuffer()**
    - **Función:** Vacía la basura de memoria del teclado para evitar ciclos de error.
    - **Recibe:** Ninguno
    - **Devuelve:** void

- **main()**
    - **Función:** Corazón del programa que controla el menú interactivo y la semilla aleatoria.
    - **Recibe:** Ninguno
    - **Devuelve:** int

4.3. Flujo de Ejecución del Sistema

1. **Inicialización:** Todo comienza en la función main(), donde el sistema arranca inicializando la semilla aleatoria (srand) usando el reloj del sistema para garantizar que el ruido generado varíe en cada ejecución.

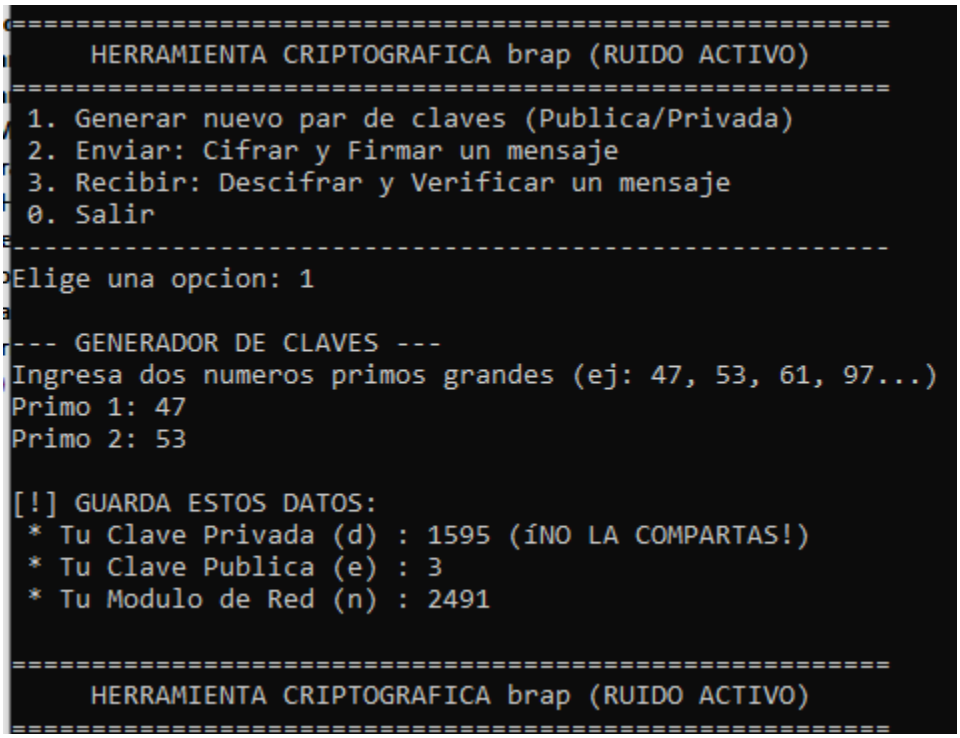
2. **Generación de Credenciales (Opción 1):** Si el usuario elige crear claves, main() solicita dos números primos y llama a generarClavesAsimetricas(). Esta función se apoya en calcularMCD() para encontrar la Clave Pública y en euclidesExtendido() para fabricar la Clave Privada, devolviendo estos valores al usuario junto con el Módulo de Red.

3. **Proceso de Emisión (Opción 2):** Cuando el usuario decide enviar un mensaje confidencial, el sistema primero llama a cifrarMensajeHex(). Dentro de ella, se genera un salto aleatorio y se usa expModular() repetidamente para cifrar tanto las letras reales como el ruido inyectado, usando aHexadecimal() para empaquetar todo en bloques exactos de 8 caracteres. Simultáneamente, el texto plano original pasa por calcularHashBBP() para obtener una huella matemática única, la cual se firma (cifra) con la clave privada del emisor usando nuevamente expModular().

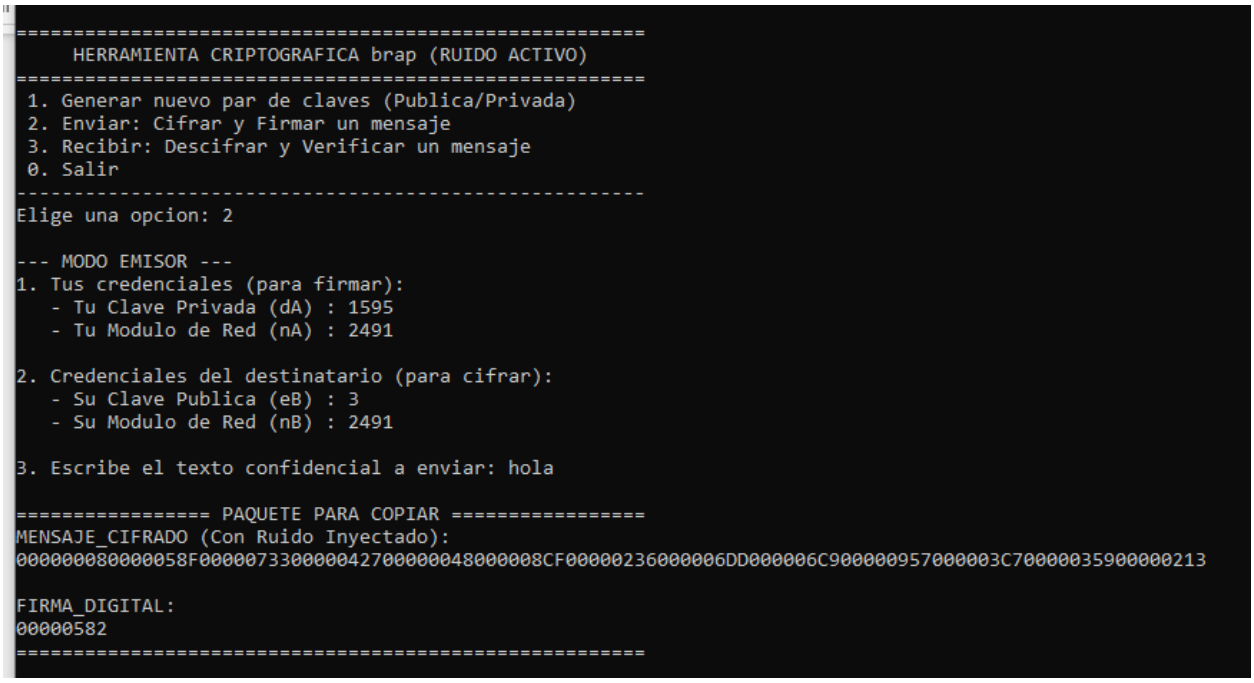
4. **Proceso de Recepción (Opción 3):** El destinatario pega el criptograma y la firma. El sistema ejecuta descifrarMensajeHex(), donde primero extrae y descifra el bloque inicial con desdeHexadecimal() y expModular() para descubrir el mapa de ruta secreto. Al dar los saltos para evadir el ruido, reconstruye el texto original. Finalmente, el sistema calcula un nuevo hash BBP del texto recuperado y lo compara con la firma digital descifrada usando la llave pública del emisor. Si ambos hashes coinciden, el ciclo se cierra validando el mensaje.

5. Pruebas

Primero se generan un nuevo par de claves



Luego usando las claves y el módulo de red se encripta un mensaje



Ahora nuevamente introduciendo las claves y el módulo de red podemos desencriptar el

mensaje

```
     HERRAMIENTA CRIPTOGRAFICA brap (RUIDO ACTIVO)
=======================================================
 1. Generar nuevo par de claves (Publica/Privada)
 2. Enviar: Cifrar y Firmar un mensaje
 3. Recibir: Descifrar y Verificar un mensaje
 0. Salir
-------------------------------------------------------
Elige una opcion: 3

--- MODO RECEPTOR ---
1. Tus credenciales (para descifrar):
   - Tu Clave Privada (dB) : 1595
   - Tu Modulo de Red (nB) : 2491

2. Credenciales de quien envia (para verificar):
   - Su Clave Publica (eA) : 3
   - Su Modulo de Red (nA) : 2491

3. Pega el MENSAJE_CIFRADO: 00000008000058F000007330000042700000048000008CF000002360000006DD000006C900000957000003C70000
035900000213
4. Pega la FIRMA_DIGITAL: 00000582

[*] PRUEBA SUPERADA: Fragmento decodificado.
    -> Camino especial encontrado: esquivando 2 bloques de ruido por caracter.

[!] PROCESANDO FIRMA DIGITAL...

[EXITO] VALIDACION PERFECTA. El mensaje es autentico y limpio de ruido.
-> CONTENIDO: "hola"
```

Observabos que el mensaje se puede cifrar y descifrar correctamente

6. Conclusión

Cumplimiento del objetivo educativo: El proyecto logró desarrollar con éxito la herramienta criptográfica "brap" en C++. Esta herramienta permite aplicar y entender de forma práctica el cifrado asimétrico mediante el uso del algoritmo matemático RSA para la generación de claves públicas y privadas.

innovación en la firma digital: Se implementó un sistema de firma digital funcional utilizando una función hash propia. Esta función fue diseñada de manera inusual inspirándose en la serie matemática BBP, la cual se usa normalmente para calcular los dígitos de Pi, con el fin de verificar que el contenido del mensaje recibido sea el original.

Ofuscación mediante "ruido activo": El sistema añade una capa adicional de seguridad combinando criptografía con esteganografía básica. Lo hace inyectando caracteres basura aleatorios (ruido) entre los datos reales del mensaje cifrado. Para que el receptor pueda leer el mensaje, primero debe descifrar un bloque oculto que le indica cuántas letras falsas debe saltar por cada letra real.

Eficacia comprobada: Las pruebas documentadas, como el envío de la palabra "hola", demostraron que el sistema funciona de principio a fin. El programa es capaz de cifrar, inyectar el ruido, descifrar esquivando correctamente los caracteres falsos y validar la firma digital.

Alcance del proyecto: El informe concluye que el objetivo de esta herramienta no es reemplazar algoritmos estándar de la industria moderna como el SHA-256. Por el contrario, su valor radica en integrar de manera funcional conceptos matemáticos complejos como números primos, máximo común divisor y aritmética modular en un solo programa educativo.